Sim-to-Real Robot Learning from Pixels with Progressive Nets

Andrei A. Rusu
DeepMind
London, UK
andreirusu@google.com

Mel Večerík
DeepMind
London, UK
matejvecerik@google.com

Thomas Rothörl
DeepMind
London, UK
tcr@google.com

Nicolas Heess
DeepMind
London, UK
heess@google.com

Razvan Pascanu
DeepMind
London, UK
razp@google.com

Raia Hadsell
DeepMind
London, UK
raia@google.com

Abstract: Applying end-to-end learning to solve complex, interactive, pixel-driven control tasks on a robot is an unsolved problem. Deep Reinforcement Learning algorithms are too slow to achieve performance on a real robot, but their potential has been demonstrated in simulated environments. We propose using *progressive networks* to bridge the reality gap and transfer learned policies from simulation to the real world. The progressive net approach is a general framework that enables reuse of everything from low-level visual features to high-level policies for transfer to new tasks, enabling a compositional, yet simple, approach to building complex skills. We present an early demonstration of this approach with a number of experiments in the domain of robot manipulation that focus on bridging the reality gap. Unlike other proposed approaches, our real-world experiments demonstrate successful task learning from raw visual input on a fully actuated robot manipulator. Moreover, rather than relying on model-based trajectory optimisation, the task learning is accomplished using only deep reinforcement learning and sparse rewards.

Keywords: Robot learning, transfer, progressive networks, sim-to-real, CoRL.

1 Introduction

Deep Reinforcement Learning offers new promise for achieving human-level control in robotics domains, especially for pixel-to-action scenarios where state estimation is from high dimensional sensors and environment interaction and feedback are critical. With deep RL, a new set of algorithms has emerged that can attain sophisticated, precise control on challenging tasks, but these accomplishments have been demonstrated primarily in simulation, rather than on actual robot platforms.

While recent advances in simulation-driven deep RL are impressive [1, 2, 3, 4, 5, 6, 7], demonstrating learning capabilities on real robots remains the bar by which we must measure the practical applicability of these methods. However, this poses a significant challenge, given the "data-hungry" training regime required for current pixel-based deep RL methods, and the relative frailty of research robots and their human handlers. One solution is to use transfer learning methods to bridge the reality gap that separates simulation from real world domains. In this paper, we use progressive networks, a deep learning architecture that has recently been proposed for transfer learning, to demonstrate such an approach, thus providing a proof-of-concept pathway by which deep RL can be used to effect fast policy learning on a real robot.

Progressive nets have been shown to produce positive transfer between disparate tasks such as Atari games by utilizing lateral connections to previously learnt models [8]. The addition of new capacity for each new task allows specialized input features to be learned, an important advantage for deep RL algorithms which are improved by sharply-tuned perceptual features. An advantage of progressive

1st Conference on Robot Learning (CoRL 2017), Mountain View, United States.

使用渐进网络从模拟到真实的机器人从像素学习

Andrei A. Rusu DeepMind 英国伦敦 andreirusu@google.com

Mel Večerík DeepMind 英国伦敦 matejvecerik@google.com

Thomas Rothörl DeepMind 英国伦敦 tcr@google.com

尼古拉斯·赫斯 DeepMind 英国伦敦 heess@google.com

Razvan Pascanu DeepMind 英国伦敦 razp@google.com

拉亚·哈德塞尔 DeepMind 英国伦敦 raia@google.com

摘要：应用端到端学习来解决机器人上复杂的、交互式的、像素驱动的控制任务是一个尚未解决的问题。深度强化学习算法太慢，无法在真实机器人上实现性能，但其潜力已在模拟环境中得到证明。我们建议使用 *progressive networks* 来弥合现实差距，并将学习到的策略从模拟转移到现实世界。渐进网络方法是一个通用框架，可以重用从低级视觉特征到转移到新任务的高级策略的所有内容，从而实现构建复杂技能的组合而简单的方法。我们通过机器人操纵领域的大量实验对这种方法进行了早期演示，这些实验的重点是弥合现实差距。与其他提出的方法不同，我们的现实世界实验证明了在完全驱动的机器人操纵器上从原始视觉输入成功进行任务学习。此外，任务学习不是依赖于基于模型的轨迹优化，而是仅使用深度强化学习和稀疏奖励来完成。

关键词 ds：机器人学习、迁移、渐进网络、模拟到真实、 CoRL。

1 简介

深度强化学习为在机器人领域实现人类水平的控制提供了新的希望，特别是对于像素到动作的场景，其中状态估计来自高维传感器，环境交互和反馈至关重要。通过深度强化学习，出现了一套新的算法，可以对具有挑战性的任务实现复杂、精确的控制，但这些成就主要是在模拟中而不是在实际的机器人平台上得到证明。

虽然模拟驱动的深度强化学习的最新进展令人印象深刻[1,2,3,4,5,6,7]，但在真实机器人上展示学习能力仍然是我们衡量这些方法实际适用性的标准。然而，考虑到当前基于像素的深度强化学习方法所需的“数据匮乏”训练机制，以及研究机器人及其人类操作员的相对脆弱性，这构成了重大挑战。一种解决方案是使用迁移学习方法来弥合模拟与现实世界领域之间的现实差距。在本文中，我们使用渐进式网络（最近为迁移学习提出的一种深度学习架构）来演示这种方法，从而提供了一种概念验证途径，通过该途径，深度强化学习可用于在真实机器人上实现快速策略学习。

渐进网络已被证明可以通过利用与先前学习的模型的横向连接来在不同的任务（例如 Atari 游戏）之间产生正迁移 [8]。每个新任务增加的新容量允许学习专门的输入特征，这是深度强化学习算法的一个重要优势，深度强化学习算法通过急剧调整的感知特征得到改进。进步的优点

1st Conference on Robot Learning (CoRL 2017), Mountain View, United States.

nets compared with other methods for transfer learning or domain adaptation is that multiple tasks may be learned sequentially, without needing to specify source and target tasks.

This paper presents an approach for transfer from simulation to the real robot that is proven using real-world, sparse-reward tasks. The tasks are learned using end-to-end deep RL, with RGB inputs and joint velocity output actions. First, an actor-critic network is trained in simulation using multiple asynchronous workers [6]. The network has a convolutional encoder followed by an LSTM. From the LSTM state, using a linear layer, we compute a set of discrete action outputs that control the different degrees of freedom of the simulated robot as well as the value function. After training, a new network is initialized with lateral, nonlinear connections to each convolutional and recurrent layer of the simulation-trained network. The new network is trained on a similar task on the real robot. Our initial findings show that the inductive bias imparted by the features and encoded policy of the simulation net is enough to give a dramatic learning speed-up on the real robot.

2 Transfer Learning from Simulation to Real

Our approach relies on the *progressive nets* architecture, which enables transfer learning through lateral connections which connect each layer of previously learnt network columns to each new column, thus supporting rich compositionality of features. We first summarize progressive nets, and then we discuss their application for transfer in robot domains.

2.1 Progressive Networks

Progressive networks are ideal for simulation-to-real transfer of policies in robot control domains, for multiple reasons. First, features learnt for one task may be transferred to many new tasks without destruction from fine-tuning. Second, the columns may be heterogeneous, which may be important for solving different tasks, including different input modalities, or simply to improve learning speed when transferring to the real robot. Third, progressive nets add new capacity, including new input connections, when transferring to new tasks. This is advantageous for bridging the reality gap, to accommodate dissimilar inputs between simulation and real sensors.

A progressive network starts with a single column: a deep neural network having L layers with hidden activations $h_i^{(1)} \in \mathbb{R}^{n_i}$, with n_i the number of units at layer $i \leq L$, and parameters $\Theta^{(1)}$ trained to convergence. When switching to a second task, the parameters $\Theta^{(1)}$ are "frozen" and a new column with parameters $\Theta^{(2)}$ is instantiated (with random initialization), where layer $h_i^{(2)}$ receives input from both $h_{i-1}^{(2)}$ and $h_{i-1}^{(1)}$ via lateral connections. Progressive networks can be generalized in a straightforward manner to have arbitrary network width per column/layer, to accommodate varying degrees of task difficulty, or to compile lateral connections from multiple, independent networks in an ensemble setting.

$$h_i^{(k)} = f\left(W_i^{(k)} h_{i-1}^{(k)} + \sum_{j<k} U_i^{(k:j)} h_{i-1}^{(j)}\right), \tag{1}$$

where $W_i^{(k)} \in \mathbb{R}^{n_i \times n_{i-1}}$ is the weight matrix of layer i of column k, $U_i^{(k:j)} \in \mathbb{R}^{n_i \times n_j}$ are the lateral connections from layer $i-1$ of column j, to layer i of column k and h_0 is the network input. f is an element-wise non-linearity: we use $f(x) = \max(0, x)$ for all intermediate layers.

In the standard pretrain-and-finetune paradigm, there is often an implicit assumption of "overlap" between the tasks. Finetuning is efficient in this setting, as parameters need only be adjusted slightly to the target domain, and often only the top layer is retrained. In contrast, we make no assumptions about the relationship between tasks, which may in practice be orthogonal or even adversarial. Progressive networks side-step this issue by allocating a new column, potentially with different structure or inputs, for each new task. Columns in progressive networks are free to reuse, modify or ignore previously learned features via the lateral connections.

Application to Reinforcement Learning. Although progressive networks are widely applicable, this paper focuses on their application to deep reinforcement learning. In this case, each column is trained to solve a particular Markov Decision Process (MDP): the k-th column thus defines a policy

与其他迁移学习或领域适应方法相比，Nets 的特点是可以顺序学习多个任务，而不需要指定源任务和目标任务。

本文提出了一种从模拟转移到真实机器人的方法，该方法已通过现实世界的稀疏奖励任务得到验证。这些任务是使用端到端深度强化学习、RGB 输入和联合速度输出动作来学习的。首先，使用多个异步工作人员对演员评论家网络进行模拟训练[6]。该网络有一个卷积编码器，后面是一个 LSTM。从 LSTM 状态，使用线性层，我们计算一组离散动作输出，这些输出控制模拟机器人的不同自由度以及价值函数。训练后，新网络将通过与模拟训练网络的每个卷积层和循环层的横向非线性连接进行初始化。新网络在真实机器人上接受类似任务的训练。我们的初步发现表明，模拟网络的特征和编码策略所产生的归纳偏差足以显着加速真实机器人的学习速度。

2 从模拟到现实的迁移学习

我们的方法依赖于 *progressive nets* 架构，该架构通过横向连接实现迁移学习，横向连接将先前学习的网络列的每一层连接到每个新列，从而支持丰富的特征组合性。我们首先总结渐进网络，然后讨论它们在机器人领域的迁移应用。

2.1 渐进网络

出于多种原因，渐进网络非常适合机器人控制领域中策略从模拟到真实的迁移。首先，为一项任务学到的特征可以转移到许多新任务，而不会因微调而受到破坏。其次，列可能是异构的，这对于解决不同的任务（包括不同的输入方式）可能很重要，或者只是为了提高转移到真实机器人时的学习速度。第三，渐进网络在转移到新任务时增加了新的容量，包括新的输入连接。这有利于弥合现实差距，适应模拟和真实传感器之间不同的输入。

渐进网络从单列开始：一个深度神经网络，具有带有隐藏激活 $h_i^{(1)} \in \mathbb{R}^{n_i}$ 的 L 层，其中 n_i 是层 $i \leq L$ 上的单元数量，参数 $\Theta^{(1)}$ 经过训练以收敛。当切换到第二个任务时，参数 $\Theta^{(1)}$ 被“冻结”，并且带有参数 $\Theta^{(2)}$ 的新列被实例化（随机初始化），其中层 $h_i^{(2)}$ 通过横向连接接收来自 $h_{i-1}^{(2)}$ 和 $h_{i-1}^{(1)}$ 的输入。渐进网络可以以简单的方式概括为每列/层具有任意网络宽度，以适应不同程度的任务难度，或者从集成设置中的多个独立网络编译横向连接。

$$h_i^{(k)} = f\left(W_i^{(k)} h_{i-1}^{(k)} + \sum_{j<k} U_i^{(k:j)} h_{i-1}^{(j)}\right), \tag{1}$$

其中，$W_i^{(k)} \in \mathbb{R}^{n_i \times n_{i-1}}$ 是第 k 列的第 i 层的权重矩阵，$U_i^{(k:j)} \in \mathbb{R}^{n_i \times n_j}$ 是从第 j 列的第 $i-1$ 层到第 k 列的第 i 层的横向连接，h_0 是网络输入。 f 是逐元素非线性：我们对所有中间层使用 $f(x) = \max(0, x)$。

在标准的预训练和微调范例中，任务之间通常存在“重叠”的隐含假设。在此设置中微调非常有效，因为参数只需根据目标域进行轻微调整，并且通常仅重新训练顶层。相反，我们不对任务之间的关系做出任何假设，这些关系实际上可能是正交的，甚至是对抗性的。渐进式网络通过为每个新任务分配一个可能具有不同结构或输入的新列来回避这个问题。渐进网络中的列可以通过横向连接自由重用、修改或忽略先前学习的特征。

强化学习的应用。尽管渐进网络应用广泛，但本文重点关注其在深度强化学习中的应用。在这种情况下，每一列都经过训练来解决特定的马尔可夫决策过程 (MDP)：第 k 列因此定义了一个策略

$\pi^{(k)}(a \mid s)$ taking as input a state s given by the environment, and generating probabilities over actions $\pi^{(k)}(a \mid s) := h_L^{(k)}(s)$. At each time-step, an action is sampled from this distribution and taken in the environment, yielding the subsequent state. This policy implicitly defines a stationary distribution $\rho_{\pi^{(k)}}(s, a)$ over states and actions.

2.2 Approach

The proposed approach for transfer from simulated to real robot domains is based on a progressive network with some specific changes. First, the columns of a progressive net do not need to have identical capacity or structure, and this can be an advantage in sim-to-real situations. Thus, the simulation-trained column is designed to have sufficient capacity and depth to learn the task from scratch, but the robot-trained columns have minimal capacity, to encourage fast learning and limit total parameter growth. Secondly, the layer-wise adapters proposed for progressive nets are unnecessary for the output layers of complementary sequences of tasks, so they are not used. Third, the output layer of the robot-trained column is initialised from the simulation-trained column in order to improve exploration. These architectural features are shown in Fig. 1.

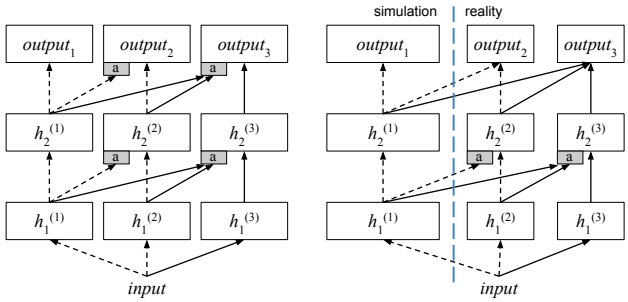

Figure 1: Depiction of a progressive network, left, and a modified progressive architecture used for robot transfer learning, right. The first column is trained on Task 1, in simulation, the second column is trained on Task 1 on the robot, and the third column is trained on Task 2 on the robot. Columns may differ in capacity, and the adapter functions (marked 'a') are not used for the output layers of this non-adversarial sequence of tasks.

The greatest risk in this approach to transfer learning is that rewards will be so sparse, or non-existent, in the real domain that the reinforcement learning will not improve a vastly suboptimal initial policy within a practical time frame. Thus, in order to maximise the likelihood of reward during exploration in the real domain, the new column is initialised such that the initial policy of the agent will be identical to the previous column. This is accomplished by initialising the weights coming from the last layer of the previous column to the output layer of the new column with the output weights of the previous column, and the connections incoming from the last hidden layer of the current column are initialised with zero-valued weights. Thus, using the example network in Fig. 1 (right), when parameters $\Theta^{(2)}$ are instantiated, layer $output_2^{(2)}$ will have input connections from $h_2^{(1)}$ and $h_2^{(2)}$. However, unlike the other parameters in $\Theta^{(2)}$, which will be randomly initialised, the weights $W_{out}^{(2)}$ will be zeros and the weights $U_{out}^{(1:2)}$ will be copied from $W_{out}^{(1)}$. Note that this only affects the initial policy of the agent and does not prevent the new column from training.

3 Related Literature

There exist many different paradigms for domain transfer and many approaches designed specifically for deep neural models, but substantially fewer approaches for transfer from simulation to reality for robot domains. Even more rare are methods that can be used for transfer in interactive, rich sensor domains using end-to-end (pixel-to-action) learning.

A growing body of work has been investigating the ability of deep networks to transfer between domains. Some research [9, 10] considers simply augmenting the target domain data with data from the source domain where an alignment exists. Building on this work, [11] starts from the observation that as one looks at higher layers in the model, the transferability of the features decreases quickly. To correct this effect, a soft constraint is added that enforces the distribution of the features to be

$\pi^{(k)}(a \mid s)$ 将环境给出的状态 s 作为输入，并生成动作 $\pi^{(k)}(a \mid s) := h_L^{(k)}(s)$ 的概率。在每个时间步，从该分布中采样一个动作并在环境中采取，产生后续状态。该策略隐式定义了状态和动作的平稳分布 $\rho_{\pi^{(k)}}(s, a)$。

2.2 方法

所提出的从模拟机器人领域转移到真实机器人领域的方法基于具有一些特定变化的渐进网络。首先，渐进网络的列不需要具有相同的容量或结构，这在模拟到真实的情况下可能是一个优势。因此，模拟训练的列被设计为具有足够的容量和深度来从头开始学习任务，但机器人训练的列具有最小的容量，以鼓励快速学习并限制总参数增长。其次，为渐进网络提出的逐层适配器对于互补任务序列的输出层来说是不必要的，因此不使用它们。第三，机器人训练列的输出层是从模拟训练列初始化的，以改进探索。这些架构特征如图 1 所示。

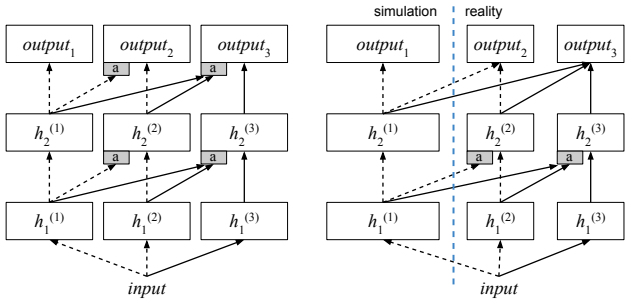

图 1：左图描绘了渐进式网络，右图描绘了用于机器人迁移学习的改进渐进式架构。第一列针对任务 1 进行训练，在模拟中，第二列针对机器人上的任务 1 进行训练，第三列针对机器人上的任务 2 进行训练。列的容量可能不同，并且适配器函数（标记为“a”）不用于此非对抗性任务序列的输出层。

这种迁移学习方法的最大风险是，在现实领域中，奖励非常稀少，甚至根本不存在，以至于强化学习无法在实际时间范围内改善大幅次优的初始策略。因此，为了在真实领域的探索过程中最大化奖励的可能性，新列被初始化，使得代理的初始策略将与前一列相同。这是通过使用前一列的输出权重初始化从前一列的最后一层到新列的输出层的权重来完成的，并且使用零值权重初始化从当前列的最后一个隐藏层传入的连接。因此，使用图 1（右）中的示例网络，当实例化参数 $\Theta^{(2)}$ 时，层 $output_2^{(2)}$ 将具有来自 $h_2^{(1)}$ 和 $h_2^{(2)}$ 的输入连接。然而，与 $\Theta^{(2)}$ 中随机初始化的其他参数不同，权重 $W_{out}^{(2)}$ 将为零，权重 $U_{out}^{(1:2)}$ 将从 $W_{out}^{(1)}$ 复制。请注意，这只会影响代理的初始策略，并不会阻止新列的训练。

3 相关文献

存在许多不同的领域转移范式和许多专门为深度神经模型设计的方法，但用于机器人领域从模拟转移到现实的方法要少得多。更罕见的是可以使用端到端（像素到动作）学习在交互式、丰富的传感器域中进行传输的方法。

越来越多的工作一直在研究深度网络在域之间传输的能力。一些研究 [9, 10] 考虑简单地使用来自存在对齐的源域的数据来增强目标域数据。在这项工作的基础上，[11]从观察到当人们观察模型中的较高层时，特征的可转移性迅速下降。为了纠正这种影响，添加了一个软约束，强制特征的分布为

more similar. In [11], a 'confusion' loss is proposed which forces the model to ignore variations in the data that separate the two domains [12, 13].

Based on [12], [14] attempts to address the simulation to reality gap by using aligned data. The work is focused on pose estimation of the robotic arm, where training happens on a triple loss that looks at aligned simulation to real data, including the domain confusion loss. The paper does not show the efficiency of the method on learning novel complex policies.

Several recent works from the supervised learning literature, e.g. [15, 16, 17], demonstrate how ideas from the adversarial training of neural networks can be used to reduce the sensitivity of a trained network to inter-domain variations, without requiring aligned training data. Intuitively these approaches train a representation that makes it hard to distinguish between data points drawn from the different domains. These ideas have, however, not yet been tested in the context of control. Demonstrating the difficulty of the problem, [10] provides evidence that a simple application of a model trained on synthetic data on the real robot fails. The paper also shows that the main failure point is the discrepancy in visual cues between simulation and reality.

Partial success on transferring from simulation to a real robot has been reported [18, 19, 20]. They focus primarily on the problem of transfer from a more restricted simpler version of a task to the full, more difficult version. While transfer from simulation to reality remains difficult, progress has been made with directly learning neural network control policies on a real robot, both from low-dimensional representations of the state and from visual input (e.g. [21],[22]). While the results are impressive, to achieve sufficient data efficiency these works currently rely on relatively restrictive task setups, specialized visual architectures, and carefully designed training regimes. Alternative approaches embrace big data ideas for robotics ([23, 24]).

4 Experiments

For training in simulation, we use the Asynchronous Advantage Actor-Critic (A3C) framework introduced in [6]. Compared to DQN [25], the model simultaneously learns a policy and a value function for predicting expected future rewards, and can be trained with CPUs, using multiple threads. A3C has been shown to converge faster than DQN, which makes it advantageous for research experimentation.

For the manipulation domain of the Jaco arm, the agent policy controls nine degrees of freedom using velocity commands. This includes six joints on the arm plus three actuated fingers. The full policy $\Pi(\mathbf{A}|s, \theta)$ comprises nine joint policies learnt by the agent, each one a softmax connected to the inputs from the previous layer and any lateral connections. Each joint policy i has three actions (a fixed positive velocity, a fixed negative velocity, and a zero velocity): $\pi_i(a_i|s; \theta_i)$. This discrete action set, while potentially lacking the precision of a continuous control policy, has worked well in practice. There is also a single value function that is linearly connected to the previous layer and lateral layers: $V(s, \theta_v)$.

We evaluate both feedforward and recurrent neural networks. Both have convolutional input layers followed by either a fully connected layer or an LSTM. A standard-sized network is used for the simulation-trained column and a reduced-capacity network is used for the robot-trained columns, chosen because we found empirically that more capacity does not accelerate learning (see Section4.2), presumably because of the features reused from the previous column. Details of the architecture are given in Figure 2 and Table 1. In all variants, the input is 3x64x64 pixels and the output is 28 (9 discrete joint policies plus one value function).

The MuJoCo physics simulator [26] is used to train the first column for our experiments, with a rendered camera view to provide observations. In the real domain, a similarly positioned RGB camera provides the input. While the modeled Jaco and its dynamics are quite accurate, the visual discrepancies are obvious, as shown in Figure 3.

The experiments are all focused around the task of reaching to a visual target, with only pure rewards provided as feedback (no shaped rewards). Though simple, this task requires that the state of the arm and the position of the target are correctly inferred from visual observations, and that the agent learns robust control over a high-dimensional state space. The arm is set to a random start position at the beginning of every episode, and the target is placed randomly within a 40cm by 30cm area. The agent receives a reward of $+1$ if its palm is within 10cm of the target, and episodes last for at most

更相似。在[11]中，提出了一种“混淆”损失，它迫使模型忽略分隔两个域的数据变化[12, 13]。

基于[12]，[14]尝试通过使用对齐数据来解决模拟与现实的差距。这项工作的重点是机器人手臂的姿态估计，其中训练发生在三重损失上，该损失着眼于与真实数据对齐的模拟，包括域混淆损失。该论文没有展示该方法在学习新颖的复杂策略方面的效率。

监督学习文献中的一些最新作品，例如[15,16,17]，展示了如何使用神经网络对抗训练的想法来降低训练网络对域间变化的敏感性，而不需要对齐的训练数据。直观上，这些方法训练的表示使得很难区分来自不同域的数据点。然而，这些想法尚未在控制环境中得到检验。[10] 证明了问题的难度，提供的证据表明，在真实机器人上简单应用基于合成数据训练的模型会失败。该论文还表明，主要的失败点是模拟与现实之间视觉线索的差异。

据报道，从模拟转移到真实机器人已取得部分成功[18,19,20]。他们主要关注从任务的更受限制的简单版本转移到完整的、更困难的版本的问题。虽然从模拟到现实的转移仍然很困难，但在真实机器人上直接学习神经网络控制策略方面已经取得了进展，无论是从状态的低维表示还是从视觉输入（例如[21]，[22]）。虽然结果令人印象深刻，但为了实现足够的数据效率，这些工作目前依赖于相对严格的任务设置、专门的视觉架构和精心设计的训练制度。替代方法包括机器人技术的大数据想法 ([23, 24])。

4 实验

对于模拟训练，我们使用[6]中介绍的异步优势 Actor-Critic (A3C) 框架。与 DQN [25] 相比，该模型同时学习策略和价值函数来预测预期的未来奖励，并且可以使用 CPU 进行训练，使用多个线程。 A3C 已被证明比 DQN 收敛得更快，这使其有利于研究实验。

对于 Jaco 臂的操纵域，代理策略使用速度命令控制九个自由度。这包括手臂上的六个关节以及三个驱动的手指。完整的策略 $\Pi(\mathbf{A}|s,\theta)$ 包含由代理学习的九个联合策略，每个策略都是连接到前一层的输入和任何横向连接的 softmax。每个联合策略 i 具有三个动作（固定正速度、固定负速度和零速度）：$\pi_i(a_i|s;\ \theta_i)$。这种离散的行动集虽然可能缺乏连续控制策略的精确性，但在实践中效果良好。还有一个与前一层和横向层线性连接的单值函数: $V(s,\theta_v)$。

我们评估前馈神经网络和循环神经网络。两者都有卷积输入层，后跟全连接层或 LSTM。标准大小的网络用于模拟训练列，而容量减少的网络用于机器人训练列，之所以选择这种网络，是因为我们根据经验发现，更大的容量并不能加速学习（参见第 4.2 节），大概是因为重复使用了上一列的特征。图 2 和表 1 给出了该架构的详细信息。在所有变体中，输入为 3x64x64 像素，输出为 28（9 个离散联合策略加上 1 个值函数）。

MuJoCo 物理模拟器 [26] 用于训练我们实验的第一列，并使用渲染的相机视图来提供观察结果。在实域中，类似位置的 RGB 相机提供输入。虽然建模的 Jaco 及其动力学相当准确，但视觉差异很明显，如图 3 所示。

这些实验都集中在达到视觉目标的任务上，仅提供纯粹的奖励作为反馈（没有成形的奖励）。虽然简单，但该任务要求从视觉观察中正确推断出手臂的状态和目标的位置，并且代理学习对高维状态空间的鲁棒控制。在每集开始时，手臂被设置为随机起始位置，目标被随机放置在 40 厘米 x 30 厘米的区域内。如果智能体的手掌距离目标 10 厘米以内，则智能体将获得 $+1$ 的奖励，并且情节最多持续

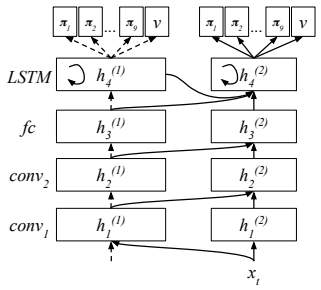

Figure 2: Detailed schematic of progressive recurrent network architecture. The activations of the LSTM are connected as inputs to the progressive column. The factored policy and single value function are shown.

	feedforward		recurrent	
	wide	narrow	wide	narrow
fc (output)	28	28	28	28
LSTM	-	-	128	16
fc	512	32	128	16
conv 2	32	8	32	8
conv 1	16	8	16	8
params	621K	39K	299K	37K

Table 1: Network sizes for wide columns (simulation-trained) and narrow columns (robot-trained). For all networks, the first convolutional layer uses 8x8, stride 4 kernels and the second uses 5x5, stride 2 kernels. The total parameters include the lateral connections.

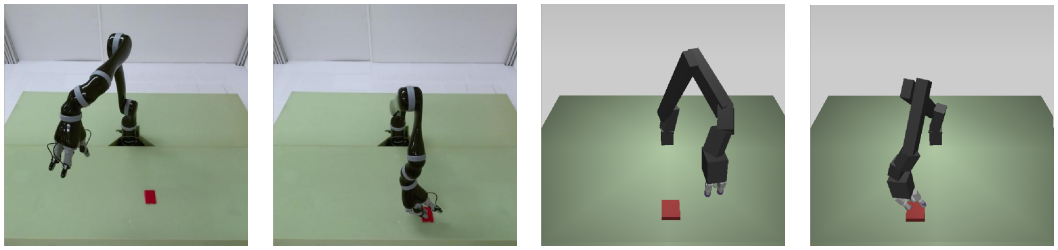

Figure 3: Sample images from the real camera input image and the MuJoCo-rendered image. Though a more realistic model appearance could have been used, the blocky Jaco model was used to accelerate MuJoCo rendering, which was done on CPUs. The images show the diversity of Jaco start positions and target positions.

50 steps. Though there is some variance due to randomized starting states, a well-performing agent can achieve an average score of over 30 points by quickly reaching to the target and remaining in safe positions at all times. The episode is terminated if the agent causes a safety violation through self-intersection, by touching the table top, or by exceeding set joint limits.

4.1 Training in simulation

The first column is trained in simulation using A3C, as previously mentioned, using a wide feedforward or recurrent network. Intuitively, it makes sense to use a larger capacity network for training in simulation, to reach maximum performance. We verified this intuition by comparing wide and narrow

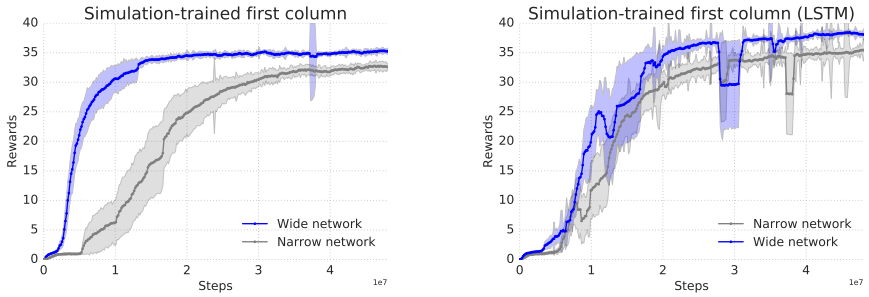

Figure 4: Learning curves are shown for wide and narrow versions of the feedforward (left) and recurrent (right) models, which are trained with the MuJoCo simulator. The plots show mean and variance over 5 training runs with different seeds and hyperparameters. Stable performance is reached after approximately 50 million steps, which is more than one million episodes. While both the feedforward and the recurrent models learn the task, the recurrent network reaches a higher final mean score.

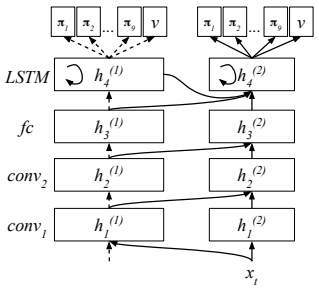

图 2：渐进式循环网络架构的详细示意图。 LSTM 的激活作为输入连接到渐进列。显示了因子策略和单值函数。

	feedforward		recurrent	
	wide	narrow	wide	narrow
fc (output)	28	28	28	28
LSTM	-	-	128	16
fc	512	32	128	16
conv 2	32	8	32	8
conv 1	16	8	16	8
params	621K	39K	299K	37K

表 1：宽列（模拟训练）和窄列（机器人训练）的网络大小。对于所有网络，第一个卷积层使用 8x8、步长 4 的内核，第二个卷积层使用 5 x5、步长 2 的内核。总参数包括横向连接。

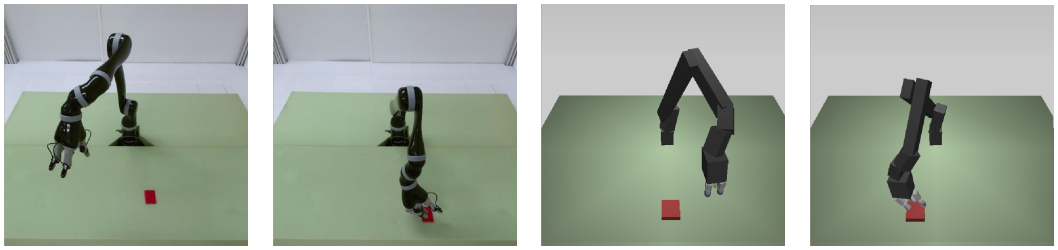

图 3：来自真实相机输入图像和 MuJoCo 渲染图像的示例图像。尽管可以使用更真实的模型外观，但块状 Jaco 模型用于加速 MuJoCo 渲染，这是在 CPU 上完成的。图像显示了 Jaco 起始位置和目标位置的多样性。

50 步。尽管由于随机的起始状态而存在一些差异，但表现良好的智能体可以通过快速到达目标并始终保持在安全位置来获得超过 30 分的平均分数。如果智能体通过自相交、触摸桌面或超出设定的关节限制而导致安全违规，则该事件将终止。

4.1 模拟训练

如前所述，第一列使用 A3C 进行模拟训练，使用宽前馈或循环网络。直观上，使用更大容量的网络进行模拟训练以达到最大性能是有意义的。我们通过比较广义和狭义来验证这一直觉

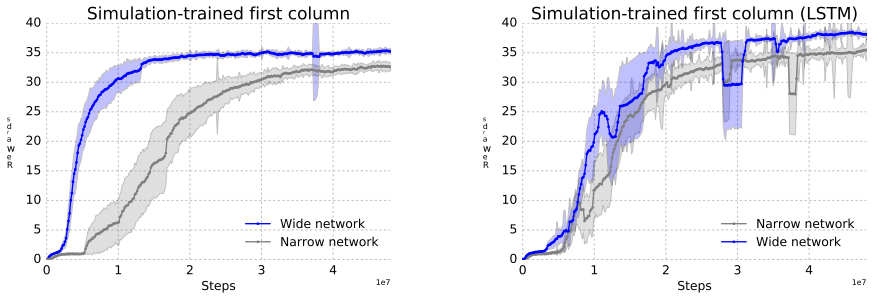

图 4：显示了宽版本和窄版本前馈（左）和循环（右）模型的学习曲线，这些模型是使用 MuJoCo 模拟器进行训练的。这些图显示了使用不同种子和超参数进行 5 次训练运行的平均值和方差。大约 5000 万步（超过 100 万集）后即可达到稳定的性能。虽然前馈模型和循环模型都学习任务，但循环网络达到了更高的最终平均分数。

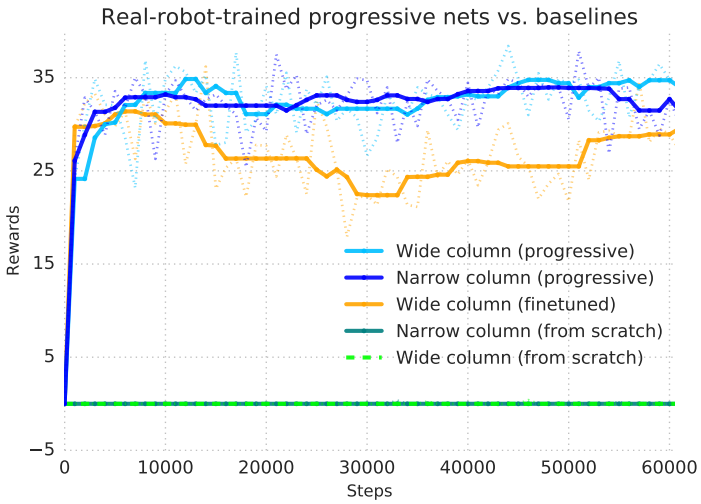

Figure 5: Real robot training: We compare progressive, finetuning, and 'from scratch' learning curves. All experiments use a recurrent architecture, trained on the robot, from RGB inputs. We compare wide and narrow columns for both the progressive experiments and the randomly initialized baseline. For all results, a median-filtered solid curve is shown overlaid on the raw rewards (dotted line). The 'from scratch' baseline was a randomly initialized narrow or wide column, both of which fail to get any reward during training.

network architectures, and found that the narrow network had slower learning and worse performance (see Figure 4). We also see that the LSTM model out-performs the feedforward model by an average of 3 points per episode. Even on this relatively simple task, full performance is only achieved after substantial interaction with the environment, on the order of 50 million steps - a number which is infeasible with a real robot.

The simulation training, compared with the real robot, is accelerated because of fast rendering, multithreaded learning algorithms, and the ability to continuously train without human involvement. We calculate that learning this task, which trains to convergence in 24 hours using a CPU compute cluster, would take 53 days on the real robot even with continuous training for 24 hours a day. Moreover, multiple experiments in parallel were used to explore hyperparameters in simulation; this sort of search would multiply the hypothetical real robot training time.

In simulation, we explore learning rates and entropy costs, which are sampled uniformly at random on a log scale. Learning rates are sampled between 5e-5 and 5e-3 and entropy costs between 1e-5 and 1e-2. The configuration with the best final performance from a grid of 30 is chosen as first column. For real Jaco experiments, both learning rates and entropy costs were optimized separately using a simulated transfer experiment with a single-threaded agent (A2C).

4.2 Transfer to the robot

To train on the real Jaco, a flat target is manually repositioned within a 40cm by 30cm area on every third episode. Rewards are given automatically by tracking the colored target and giving reward based on the position of the Jaco gripper with respect to it. We train a baseline from scratch, a finetuned first column, and a progressive second column. Each experiment is run for approximately 60000 steps (about four hours). The baseline is trained by randomly initializing a narrow network and then training. We also try a randomly initialized wide network. As seen in Figure 5 (green curve), the randomly initialized column fails to learn and the agent gets zero reward throughout training. The progressive second column gets to 34 points, while the experiment with finetuning, which starts with the simulation-trained column and continues training on the robot, does not reach the same score as the progressive network.

Finetuning vs. progressive approaches. The progressive approach is clearly well-suited for continual learning scenarios, where it is important to mitigate forgetting of previous tasks while supporting transfer to new tasks, but the advantage is less intuitive for curricula of tasks where the focus is on

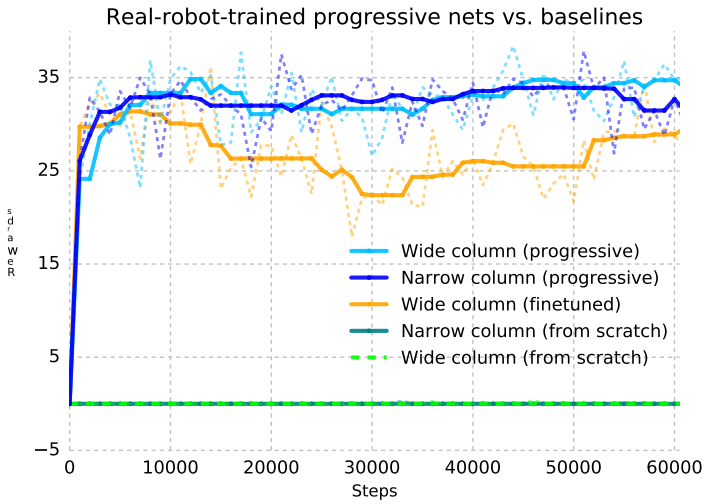

图 5：真实的机器人训练：我们比较渐进式、微调式和“从头开始”的学习曲线。所有实验都使用循环架构，根据 RGB 输入在机器人上进行训练。我们比较渐进实验和随机初始化基线的宽柱和窄柱。对于所有结果，中值过滤的实线显示在原始奖励上（虚线）。 “从头开始”基线是随机初始化的窄列或宽列，两者在训练过程中都无法获得任何奖励。

网络架构，发现窄网络学习速度较慢，性能较差（见图 4）。我们还发现 LSTM 模型每集平均优于前馈模型 3 个点。即使在这个相对简单的任务中，只有在与环境进行大量交互后才能实现全部性能，大约 5000 万步——这个数字对于真正的机器人来说是不可行的。

与真实机器人相比，模拟训练速度更快，因为快速渲染、多线程学习算法以及无需人工参与即可连续训练的能力。我们计算出，学习这项使用 CPU 计算集群在 24 小时内训练收敛的任务，即使每天连续训练 24 小时，在真实机器人上也需要 53 天。此外，还使用多个并行实验来探索模拟中的超参数；这种搜索会增加假设的真实机器人训练时间。

在模拟中，我们探索学习率和熵成本，它们是在对数尺度上随机均匀采样的。学习率在 5e-5 和 5e-3 之间采样，熵成本在 1e-5 和 1e-2 之间采样。从 30 个网格中选择具有最佳最终性能的配置作为第一列。对于真实的 Jaco 实验，使用单线程代理 (A2C) 的模拟传输实验分别优化了学习率和熵成本。

4.2 传输至机器人

为了在真实的 Jaco 上进行训练，每第三集手动将平面目标重新定位在 40 厘米 x 30 厘米的区域内。通过跟踪彩色目标并根据 Jaco 夹具相对于它的位置给予奖励，自动给予奖励。我们从头开始训练基线、微调的第一列和渐进的第二列。每个实验运行大约 60000 个步骤（大约四个小时）。通过随机初始化一个狭窄的网络然后进行训练来训练基线。我们还尝试随机初始化的宽网络。如图 5（绿色曲线）所示，随机初始化的列无法学习，并且代理在整个训练过程中获得零奖励。渐进式第二列得到了 34 分，而微调实验（从模拟训练列开始并继续在机器人上进行训练）并未达到与渐进式网络相同的分数。

微调与渐进方法。渐进式方法显然非常适合持续学习场景，在这种情况下，在支持转移到新任务的同时，减少对先前任务的遗忘很重要，但对于重点关注的任务课程来说，优点不太直观。

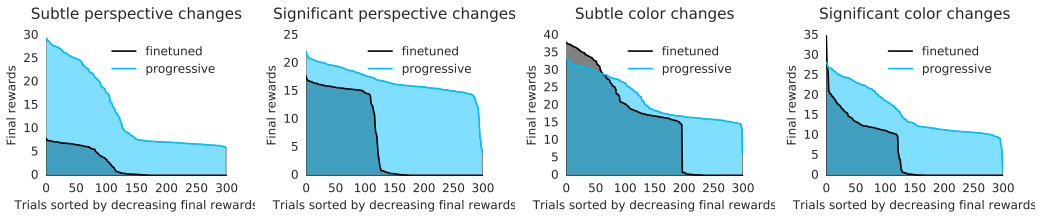

Figure 6: To analyse the relative stability and performance of finetuning vs. progressive approaches, we add color or perspective changes to the environment in simulation and then train 300 networks with different random seeds, learning rates, and entropy costs. The progressive networks have significantly higher performance and less sensitivity to hyperparameter selection for all four experiments.

maximising transfer learning. To assess this empirically, we start with a simulator-trained first column, as described above, and then either finetune that column or add a narrow progressive column and retrain for the reacher task under a variety of conditions, including small or large color changes and small or large perspective changes. For each of these environment perturbations, we train 300 times with different seeds, learning rates, and entropy costs, which are the most sensitive hyperparameters. As shown in Figure 6, we find that progressive networks are more stable and reach higher final performance than finetuning.

4.3 Transfer to a dynamic robot task with proprioception

Unlike the finetuning paradigm, which is unable to accommodate changing network morphology or new input modalities, progressive nets offer a flexibility that is advantageous for transferring to new data sources while still leveraging previous knowledge. To demonstrate this, we train a second column on the reacher task but add proprioceptive features as an additional input, alongside the RGB images. The proprioceptive features are joint angles and velocities for each of the 9 joints of the arm and fingers, 18 in total, input to a MLP (a single linear layer plus ReLU) and joined with the outputs of the convolutional stack. Then, a third progressive column is added that only learns from the proprioceptive features, while the visual input is forwarded through the previous columns and the features are used via the lateral connections. A diagram of this architecture is shown in Figure 7 (left).

To evaluate this architecture, we train on a dynamic target task. By employing a small motorized pulley, the red target is smoothly translated across the table with random reversals in the motion, creating a tracking task that requires a different control policy while maintaining a similar visual presentation. Other aspects of the task, including rewards and episode lengths, were kept the same. If the second column is trained on this *conveyor* task, the learning is relatively slow, and full performance is reached after 50000 steps (about 4 hours). If the second column is instead trained on the static reacher task, and the third column is then trained on the conveyor task, we observe immediate transfer, and full performance is reached almost immediately (Figure 7, right). This demonstrates both the utility of progressive nets for curriculum tasks, as well as the capability of the architecture to immediately reuse previously learnt features.

5 Discussion

Transfer learning, the ability to accumulate and transfer knowledge to new domains, is a core characteristic of intelligent beings. *Progressive neural networks* offer a framework that can be used for continual learning of many tasks and which facilitates transfer learning, even across the divide which separates simulation and robot. We took full advantage of the flexibility and computational scaling afforded by simulation and compared many hyperparameters and architectures for a random start, random target control task with visual input, then successfully transferred the skill to an agent training on the real robot.

In order to fulfill the potential of deep reinforcement learning applied in real-world robotic domains, learning needs to become many times more efficient. One route to achieving this is via transfer learning from simulation-trained agents. We have described an initial set of experiments that prove that progressive nets can be used to achieve reliable, fast transfer for pixel-to-action RL policies.

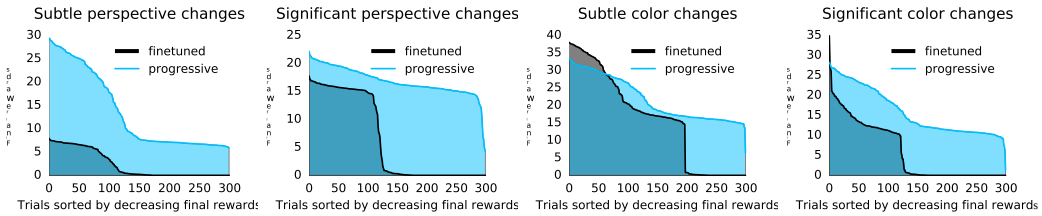

图 6：为了分析微调与渐进方法的相对稳定性和性能，我们在模拟中向环境添加颜色或视角变化，然后使用不同的随机种子、学习率和熵成本训练 300 个网络。对于所有四个实验，渐进式网络具有显着更高的性能，并且对超参数选择的敏感性较低。

最大化迁移学习。为了根据经验评估这一点，我们从模拟器训练的第一列开始，如上所述，然后微调该列或添加一个狭窄的渐进列，并在各种条件下重新训练到达者任务，包括小或大的颜色变化以及小或大的视角变化。对于每个环境扰动，我们使用不同的种子、学习率和熵成本（这些是最敏感的超参数）训练 300 次。如图 6 所示，我们发现渐进网络比微调网络更稳定，并且能达到更高的最终性能。

4.3 转移到具有本体感觉的动态机器人任务

与无法适应不断变化的网络形态或新的输入方式的微调范式不同，渐进网络提供了一种灵活性，有利于转移到新的数据源，同时仍然利用以前的知识。为了证明这一点，我们在接触器任务上训练第二列，但在 RGB 图像旁边添加本体感受特征作为附加输入。本体感受特征是手臂和手指的 9 个关节（总共 18 个）中每个关节的关节角度和速度，输入到 MLP（单个线性层加 ReLU）并与卷积堆栈的输出相连接。然后，添加第三个渐进列，仅从本体感受特征中学习，而视觉输入通过前面的列转发，并且通过横向连接使用特征。图 7（左）显示了该架构的示意图。

为了评估这种架构，我们对动态目标任务进行训练。通过采用小型电动滑轮，红色目标可以在运动中随机反转地平滑地穿过桌子，从而创建需要不同控制策略同时保持类似视觉呈现的跟踪任务。任务的其他方面，包括奖励和剧集长度，保持不变。如果第二列在此 *conveyor* 任务上进行训练，则学习速度相对较慢，并且在 50000 步（大约 4 小时）后达到完整性能。如果第二列接受静态到达器任务的训练，然后第三列接受传送带任务的训练，我们会观察到立即转移，并且几乎立即达到全部性能（图 7，右）。这证明了渐进网络在课程任务中的实用性，以及该架构立即重用先前学习的功能的能力。

5 讨论

迁移学习，即积累知识并将其迁移到新领域的能力，是智能生物的核心特征。
Progressive neural networks 提供了一个框架，可用于持续学习许多任务，并促进迁移学习，甚至跨越模拟和机器人之间的鸿沟。我们充分利用模拟提供的灵活性和计算规模，并比较了随机启动、具有视觉输入的随机目标控制任务的许多超参数和架构，然后成功地将技能转移到在真实机器人上进行训练的代理。

为了发挥深度强化学习在现实世界机器人领域的应用潜力，学习需要变得更加高效。实现这一目标的一种途径是通过经过模拟训练的代理进行迁移学习。我们描述了一组初始实验，证明渐进网络可用于实现像素到动作 RL 策略的可靠、快速传输。

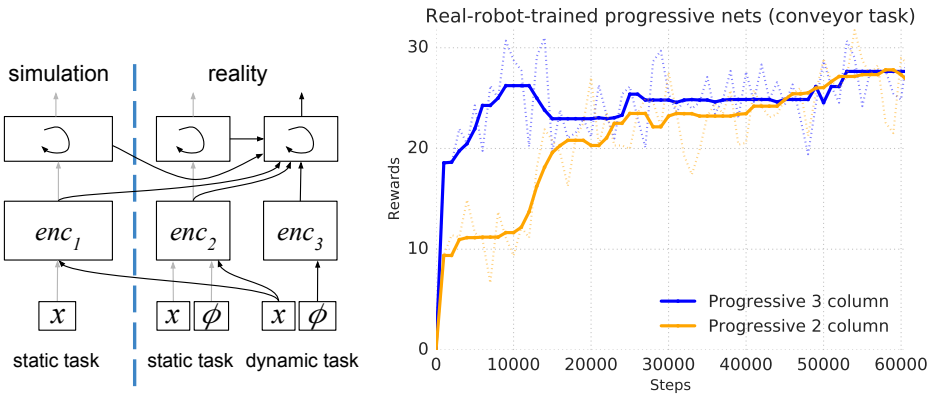

Figure 7: Real robot training results are shown for the dynamic 'conveyor' task. A three-column architecture is depicted (left), in which vision (x) is used to train column one, vision and proprioception (ϕ) are used in column two, and only proprioception is used to train column three. Encoder 1 is a convolutional net, encoder 2 is a convolutional net with proprioceptive features added before the LSTM, and encoder 3 is an MLP. The learning curves (right) show the results of training on a conveyor (dynamic target) task. If the conveyor task is learned as the third column, rather than the second, then the learning is significantly faster.

References

[1] S. Levine and P. Abbeel. Learning neural network policies with guided policy search under unknown dynamics. In Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27*, pages 1071–1079. Curran Associates, Inc., 2014. URL http://papers.nips.cc/paper/5444-learning-neural-network-policies-with-guided-policy-search-under-unknown-dynamics.pdf.

[2] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015.

[3] N. Heess, G. Wayne, D. Silver, T. P. Lillicrap, T. Erez, and Y. Tassa. Learning continuous control policies by stochastic value gradients. In *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, pages 2944–2952, 2015. URL http://papers.nips.cc/paper/5796-learning-continuous-control-policies-by-stochastic-value-gradients.

[4] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016. URL http://arxiv.org/abs/1509.02971.

[5] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

[6] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Int'l Conf. on Machine Learning (ICML)*, 2016.

[7] S. Gu, T. P. Lillicrap, I. Sutskever, and S. Levine. Continuous deep q-learning with model-based acceleration. In *ICML 2016*, 2016.

[8] A. Rusu, N. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[9] X. Peng, B. Sun, K. Ali, and K. Saenko. Learning deep object detectors from 3d models. In *2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015*, pages 1278–1286, 2015.

[10] H. Su, C. R. Qi, Y. Li, and L. J. Guibas. Render for CNN: viewpoint estimation in images using cnns trained with rendered 3d model views. In *2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015*, pages 2686–2694, 2015.

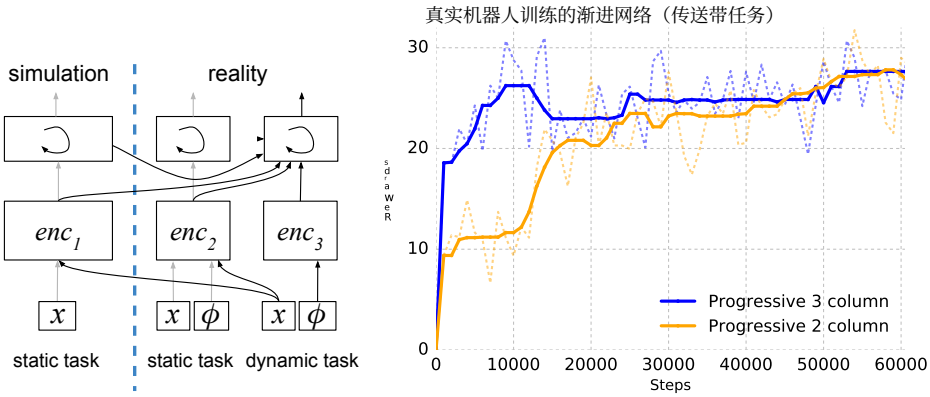

图 7：显示了动态“传送带”任务的真实机器人训练结果。描绘了三列架构（左），其中视觉（x）用于训练第一列，视觉和本体感觉（ϕ）用于第二列，仅本体感觉用于训练第三列。编码器 1 是一个卷积网络，编码器 2 是一个在 LSTM 之前添加了本体感受特征的卷积网络，编码器 3 是一个 MLP。学习曲线（右）显示了传送带（动态目标）任务的训练结果。如果传送带任务是作为第三列而不是第二列来学习的，那么学习速度会明显加快。

参考

[1] S. Levine 和 P. Abbeel。在未知动态下通过引导策略搜索来学习神经网络策略。载于 Z. Ghahramani、M. Welling、C. Cortes、N. D. Lawrence 和 K. Q. Weinberger，编辑，*Advances in Neural Information Processing Systems 27*，第 1071-1079 页。Curran Associates, Inc.，2014。URL http://papers.nips.cc/paper/5444-learning-neural-network-policies-with-guided-policy-search-under-unknown-dyn

艾米斯。

pdf。[2] J. Schulman、S. Levine、P. Moritz、M. I. Jordan 和 P. Abbeel。信托区域政策优化。*Proceedings of the 32nd International Conference on Machine Learning (ICML)*，2015 年。[3] N. Heess、G. Wayne、D. Silver、T. P. Lillicrap、T. Erez 和 Y. Tassa。通过随机值梯度学习连续控制策略。*Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*，第 2944–2952 页，2015 年。URL http://papers.nips.cc/paper/5796-learning-continuous-control-policies-by-stochastic-value-gradients。[4] T. P. Lillicrap、J. J. Hunt、A. Pritzel、N. Heess、T. Erez、Y. Tassa、D. Silver 和 D. Wierstra。通过深度强化学习进行持续控制。*Proceedings of the International Conference on Learning Representations (ICLR)*，2016。网址http://arxiv.org/abs/1509.02971。[5] J. Schulman、P. Moritz、S. Levine、M. Jordan 和 P. Abbeel。使用广义优势估计的高维连续控制。*Proceedings of the International Conference on Learning Representations (ICLR)*，2016 年。[6] V. Mnih、A. P. Badia、M. Mirza、A. Graves、T. P. Lillicrap、T. Harley、D. Silver 和 K. Kavukcuoglu。深度强化学习的异步方法。*Int'l Conf. on Machine Learning (ICML)*，2016 年。[7] S. Gu、T. P. Lillicrap、I. Sutskever 和 S. Levine。具有基于模型的加速的持续深度 q 学习。*ICML 2016*，2016 年。[8] A. Rusu、N. Rabinowitz、G. Desjardins、H. Soyer、J. Kirkpatrick、K. Kavukcuoglu、R. Pascanu 和 R. Hadsell。渐进神经网络。*arXiv preprint arXiv:1606.04671*，2016。[9] X. Peng、B. Sun、K. Ali 和 K. Saenko。从 3D 模型学习深度物体检测器。见 *2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015*，第 1278–1286 页，2015 年。[10] H. Su、C. R. Qi、Y. Li 和 L. J. Guibas。CNN 渲染：使用经过渲染 3D 模型视图训练的 CNN 在图像中进行视点估计。*2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015*，第 2686-2694 页，2015 年。

[11] M. Long, Y. Cao, J. Wang, and M. I. Jordan. Learning transferable features with deep adaptation networks. In *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, pages 97–105, 2015.

[12] E. Tzeng, J. Hoffman, T. Darrell, and K. Saenko. Simultaneous deep transfer across domains and tasks. In *2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015*, pages 4068–4076, 2015.

[13] E. Tzeng, J. Hoffman, N. Zhang, K. Saenko, and T. Darrell. Deep domain confusion: Maximizing for domain invariance. *CoRR*, abs/1412.3474, 2014. URL http://arxiv.org/abs/1412.3474.

[14] E. Tzeng, C. Devin, J. Hoffman, C. Finn, X. Peng, S. Levine, K. Saenko, and T. Darrell. Towards adapting deep visuomotor representations from simulated to real environments. *CoRR*, abs/1511.07111, 2015. URL http://arxiv.org/abs/1511.07111.

[15] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35, 2016.

[16] H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, and M. Marchand. Domain-adversarial neural networks. *CoRR*, abs/1412.4446, 2014. URL http://arxiv.org/abs/1412.4446.

[17] K. Bousmalis, G. Trigeorgis, N. Silberman, D. Krishnan, and D. Erhan. Domain separation networks. In *Advances in Neural Information Processing Systems*, pages 343–351, 2016.

[18] S. Barrett, M. E. Taylor, and P. Stone. Transfer learning for reinforcement learning on a physical robot. In *Ninth International Conference on Autonomous Agents and Multiagent Systems - Adaptive Learning Agents Workshop (AAMAS - ALA)*, 2010.

[19] S. James and E. Johns. 3D Simulation for Robot Arm Control with Deep Q-Learning. *ArXiv e-prints*, 2016.

[20] Y. Zhu, R. Mottaghi, E. Kolve, J. J. Lim, A. Gupta, L. Fei-Fei, and A. Farhadi. Target-driven visual navigation in indoor scenes using deep reinforcement learning. In *Robotics and Automation (ICRA), 2017 IEEE International Conference on*, pages 3357–3364. IEEE, 2017.

[21] S. Levine, C. Finn, T. Darrell, and P. Abbeel. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016.

[22] S. Levine, N. Wagener, and P. Abbeel. Learning contact-rich manipulation skills with guided policy search. In *IEEE International Conference on Robotics and Automation, ICRA 2015, Seattle, WA, USA, 26-30 May, 2015*, pages 156–163, 2015.

[23] L. Pinto and A. Gupta. Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours. In *ICRA 2016*, 2016.

[24] S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International Journal of Robotics Research*, page 0278364917710318, 2016.

[25] V. Mnih, K. Kavukcuoglu, D. Silver, A. Rusu, J. Veness, M. Bellemare, A. Graves, M. Riedmiller, A. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[26] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *International Conference on Intelligent Robots and Systems IROS*, 2012.

[11] M. Long，Y. Cao，J. Wang，M. I. Jordan。通过深度适应网络学习可转移特征。见 *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*，第 97-105 页，2015 年。 [12] E. Tzeng、J. Hoffman、T. Darrell 和 K. Saenko。跨领域和任务的同步深度传输。 *2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015* ，第 4068–4076 页，2015 年。 [13] E. Tzeng、J. Hoffman、N. Zhang、K. Saenko 和 T. Darrell。深度域混淆：域不变性最大化。 *CoRR*，abs/1412.3474，2014。网址http://arxiv.org/abs/1412.3474。 [14] E. Tzeng、C. Devin、J. Hoffman、C. Finn、X. Peng、S. Levine、K. Saenko 和 T. Darrell。致力于将深度视觉运动表征从模拟环境适应到真实环境。 *CoRR*，abs/1511.07111，2015。网址http://arxiv.org/abs/1511.07111。 [15] Y. Ganin、E. Ustinova、H. Ajakan、P. Germain、H. Larochelle、F. Laviolette、M. Marchand 和 V. Lempitsky。神经网络的领域对抗训练。 *Journal of Machine Learning Research*, 17(59):1–35, 2016。 [16] H. Ajakan、P. Germain、H. Larochelle、F. Laviolette 和 M. Marchand。领域对抗性神经网络。 *CoRR*，abs/1412.4446，2014。网址http://arxiv.org/abs/1412.4446。 [17] K. Bousmalis、G. Trigeorgis、N. Silberman、D. Krishnan 和 D. Erhan。域分离网络。见 *Advances in Neural Information Processing Systems* ，第 343-351 页，2016 年。 [18] S. Barrett、M. E. Taylor 和 P. Stone。用于物理机器人强化学习的迁移学习。 *Ninth International Conference on Autonomous Agents and Multiagent Systems - Adaptive Learning Agents Workshop (AAMAS - ALA)*，2010 年。[19] S. James 和 E. Johns。通过深度 Q 学习进行机器人手臂控制的 3D 仿真。 *ArXiv e-prints*，2016。[20] Y. Zhu、R. Mottaghi、E. Kolve、J. J. Lim、A. Gupta、L. Fei-Fei 和 A. Farhadi。使用深度强化学习在室内场景中进行目标驱动的视觉导航。在 *Robotics and Automation (ICRA), 2017 IEEE International Conference on*，第 3357-3364 页。 IEEE，2017。 [21] S. Levine、C. Finn、T. Darrell 和 P. Abbeel。深度视觉运动策略的端到端训练。 *Journal of Machine Learning Research*，17(39):1–40，2016。 [22] S. Levine、N. Wagener 和 P. Abbeel。通过引导策略搜索来学习接触丰富的操作技能。 *IEEE International Conference on Robotics and Automation, ICRA 2015, Seattle, WA, USA, 26-30 May, 2015*，第 156-163 页，2015 年。[23] L. Pinto 和 A. Gupta。超大型自我监督：从 50k 次尝试和 700 个机器人小时中学习抓取。 *ICRA 2016*，2016 年。[24] S. Levine、P. Pastor、A. Krizhevsky、J. Ibarz 和 D. Quillen。通过深度学习和大规模数据收集来学习机器人抓取的手眼协调。 *The International Journal of Robotics Research*，第 0278364917710318 页，2016。 [25] V. Mnih、K. Kavukcuoglu、D. Silver、A. Rusu、J. Veness、M. Bellemare、A. Graves、M. Riedmiller、A. Fidjeland、G. Ostrovski、S. Petersen、C. Beattie、A. Sadik、I. Antonoglou、H. King、D. Kumaran、D. Wierstra、S. Legg 和 D. Hassabis。通过深度强化学习实现人类水平的控制。 *Nature*, 518(7540):529–533, 2015。 [26] E. Todorov、T. Erez 和 Y. Tassa。 Mujoco：用于基于模型的控制的物理引擎。在 *International Conference on Intelligent Robots and Systems IROS*，2012 年。